

Green Hydrogen Production Monitoring & Analysis System

Introduction

Green hydrogen is a clean and eco-friendly energy source produced using renewable energy such as solar and wind power. Monitoring the hydrogen production process is important to improve efficiency, reduce energy wastage, and maintain safe operation. This project focuses on developing a software-based system that monitors and analyses green hydrogen production data in real time through an interactive dashboard and analytical tools.

Abstract

The Green Hydrogen Production Monitoring & Analysis System is a software application designed to monitor, analyze, and visualize the process of green hydrogen production in real time. The system provides a digital platform that tracks important production parameters and displays them through graphs, charts, and dashboards for better understanding and analysis.

The application continuously monitors various parameters such as:

- Temperature
- Water level
- Pressure
- Hydrogen production rate
- Energy consumption

The collected data is processed and stored in a database for future analysis and report generation. The system also includes visualization features that help users understand production efficiency and identify abnormal conditions during operation.

Main Features:

- Real-time monitoring dashboard
- Live data visualization using charts and graphs
- Production analysis and efficiency tracking
- Alert system for unusual parameter changes
- Data storage and report generation.
- User-friendly web interface

The project is developed using **Python full-stack technologies**. The frontend is designed using HTML, CSS, and JavaScript, while the backend is implemented using Python Flask and a database system for data management.

This project demonstrates how software technologies can be used for smart renewable energy monitoring and analysis. It provides an efficient and cost-effective solution for understanding and managing green hydrogen production systems.

Project Objectives:

- To develop a software-based system for monitoring green hydrogen production in real time.
- To track important production parameters such as temperature, voltage, current, pressure, water level, and hydrogen generation rate.
- To analyze production data and evaluate the efficiency of the hydrogen generation process.
- To provide a user-friendly dashboard with graphs and charts for easy visualization of data.
- To store production data in a database for future analysis and report generation.
- To detect abnormal conditions and generate alerts for safe and efficient operation.

Advantages of the project:

- Provides real-time monitoring of green hydrogen production.
- Improves production efficiency through data analysis.
- Reduces manual monitoring effort.
- Displays data using graphs and dashboards.
- Detects abnormal conditions and generates alerts.
- Stores production data for future analysis and reports.
- Supports renewable and sustainable energy management.
- User-friendly and cost-effective solution.

CODE:

App.py(main code because it routes all the files)

```
import os, sqlite3, random
```

```
from datetime import datetime, timedelta
```

```
from flask import Flask, render_template, request, redirect, flash, jsonify
```

```
app = Flask(__name__)
```

```
app.secret_key = 'h2_secret'
```

```
DB = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'database.db')
```

```
def get_db():
```

```
    conn = sqlite3.connect(DB)
```

```
    conn.row_factory = sqlite3.Row
```

```
    return conn
```

```
def init_db():
```

```
    with get_db() as c:
```

```
        c.execute("""CREATE TABLE IF NOT EXISTS production_data (

            id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT, power REAL,
operating_time REAL,

            energy_consumed REAL, temperature REAL, pressure REAL, hydrogen_rate REAL,

            hydrogen_produced REAL, water_consumption REAL, specific_energy_consumption
REAL)""")
```

```
@app.route('/')
```

```

@app.route('/dashboard')

def dashboard():

    with get_db() as c:

        logs = c.execute("SELECT * FROM production_data ORDER BY timestamp
DESC").fetchall()

        return render_template('dashboard.html', latest=logs[0] if logs else None,
history_json=[dict(l) for l in reversed(logs)])

@app.route('/add', methods=['GET', 'POST'])

def add_data():

    if request.method == 'POST':

        try:

            p = float(request.form['power'])

            t = float(request.form['operating_time'])

            h = float(request.form['hydrogen_rate'])

            w = float(request.form['water_consumption'])

            temp = float(request.form['temperature'])

            press = float(request.form['pressure'])

            if any(x < 0 for x in [p, t, h, w, temp, press]) or t == 0 or h == 0:

                flash("Error: Invalid inputs.", "danger")

                return render_template('add_data.html')

            e, prod = p * t, h * t

            sec = p / h if h > 0 else 0

            ts = request.form.get('timestamp') or datetime.now().strftime('%Y-%m-%d
%H:%M:%S')

```

```

with get_db() as c:

    c.execute("INSERT INTO production_data (timestamp, power, operating_time,
energy_consumed,    temperature,    pressure,    hydrogen_rate,    hydrogen_produced,
water_consumption, specific_energy_consumption) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",

            (ts, p, t, e, temp, press, h, prod, w, sec))

    flash("Reading saved!", "success")

    return redirect('/dashboard')

except ValueError:

    flash("Error: Numeric values required.", "danger")

return render_template('add_data.html')


@app.route('/simulate-tick', methods=['POST'])

def simulate_tick():

    with get_db() as c:

        last = c.execute("SELECT * FROM production_data ORDER BY timestamp DESC
LIMIT 1").fetchone()

        p, h, temp, press, w = (last['power'], last['hydrogen_rate'], last['temperature'],
last['pressure'], last['water_consumption']) if last else (83.9, 16.04, 63.2, 29.6, 144.4)

        p = max(75.0, min(95.0, p + random.uniform(-1.0, 1.0)))

        h = max(14.0, min(18.0, h + random.uniform(-0.15, 0.15)))

        temp = max(60.0, min(68.0, temp + random.uniform(-0.25, 0.25)))

        press = max(28.0, min(32.0, press + random.uniform(-0.15, 0.15)))

        w = max(130.0, min(150.0, w + random.uniform(-0.8, 0.8)))

        t, ts = 0.1, datetime.now().strftime('%Y-%m-%d %H:%M:%S')

        e, prod = p * t, h * t

```

```
sec = p / h
```

```
c.execute("INSERT INTO production_data (timestamp, power, operating_time,  
energy_consumed, temperature, pressure, hydrogen_rate, hydrogen_produced,  
water_consumption, specific_energy_consumption) VALUES (?,?,?,?,?,?,?,?,?)",
```

```
(ts, p, t, e, temp, press, h, prod, w, sec))
```

```
logs = c.execute("SELECT * FROM production_data ORDER BY timestamp  
DESC").fetchall()
```

```
return jsonify(latest={'timestamp': ts, 'power': p, 'operating_time': t, 'energy_consumed': e,  
'temperature': temp, 'pressure': press, 'hydrogen_rate': h, 'hydrogen_produced': prod,  
'water_consumption': w, 'specific_energy_consumption': sec},
```

```
history=[dict(l) for l in reversed(logs)])
```

```
@app.route('/reset-simulation', methods=['POST'])
```

```
def reset_simulation():
```

```
    with get_db() as c:
```

```
        c.execute("DELETE FROM production_data")
```

```
    flash("Simulation reset.", "success")
```

```
    return redirect('/dashboard')
```

```
@app.route('/analysis')
```

```
def analysis():
```

```
    with get_db() as c:
```

```
        logs = c.execute("SELECT * FROM production_data").fetchall()
```

```
    if not logs: return render_template('analysis.html', stats=None)
```

```
    n = len(logs)
```

```
    e_tot = sum(l['energy_consumed'] for l in logs)
```

```

h_tot = sum(l['hydrogen_produced'] for l in logs)

latest = logs[-1]

return render_template('analysis.html', stats={

    'total_energy': round(e_tot, 2), 'total_hydrogen': round(h_tot, 2), 'avg_sec': round(e_tot /
h_tot if h_tot > 0 else 0, 2),

    'avg_temp': round(sum(l['temperature'] for l in logs) / n, 1), 'avg_pressure':
round(sum(l['pressure'] for l in logs) / n, 1), 'avg_h2_rate': round(sum(l['hydrogen_rate'] for l
in logs) / n, 3),

    'h2_trend': "Increasing" if n >= 2 and latest['hydrogen_rate'] > logs[-2]['hydrogen_rate']
else "Stable",

    'energy_trend': "Increasing" if n >= 2 and latest['energy_consumed'] > logs[-
2]['energy_consumed'] else "Stable",

    'temp_status': "Optimal (50°C - 70°C)" if 50 <= latest['temperature'] <= 70 else "Outside
Demo Range",

    'press_status': "Optimal (10 - 35 bar)" if 10 <= latest['pressure'] <= 35 else "Outside Demo
Range"

})

```

```

@app.route('/history')

```

```

def history():

```

```

    with get_db() as c:

```

```

        logs = c.execute("SELECT * FROM production_data ORDER BY timestamp
DESC").fetchall()

```

```

    return render_template('history.html', logs=logs)

```

```

@app.route('/about')

```

```

def about(): return render_template('about.html')

```

```

@app.route('/load-demo-data', methods=['POST'])

def load_demo_data():

    with get_db() as c:

        c.execute("DELETE FROM production_data")

        now = datetime.now()

        for i, (p, h, temp, press, w) in enumerate([(78.0, 15.0, 61.0, 29.0, 135.0), (80.5, 15.4, 62.5,
29.5, 138.0), (82.0, 15.8, 63.0, 30.0, 141.0), (83.9, 16.04, 63.2, 29.6, 144.4), (85.2, 16.3, 64.0,
30.2, 146.5)]):

            ts = (now - timedelta(hours=(8 - 2 * i))).strftime("%Y-%m-%d %H:%M:%S")

            c.execute("INSERT INTO production_data (timestamp, power, operating_time,
energy_consumed, temperature, pressure, hydrogen_rate, hydrogen_produced,
water_consumption, specific_energy_consumption) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",

                (ts, p, 2.0, p * 2.0, temp, press, h, h * 2.0, w, p / h))

            flash("Demo data loaded.", "success")

    return redirect('/dashboard')


if __name__ == '__main__':

    init_db()

    app.run(debug=True)

```


Outputs:

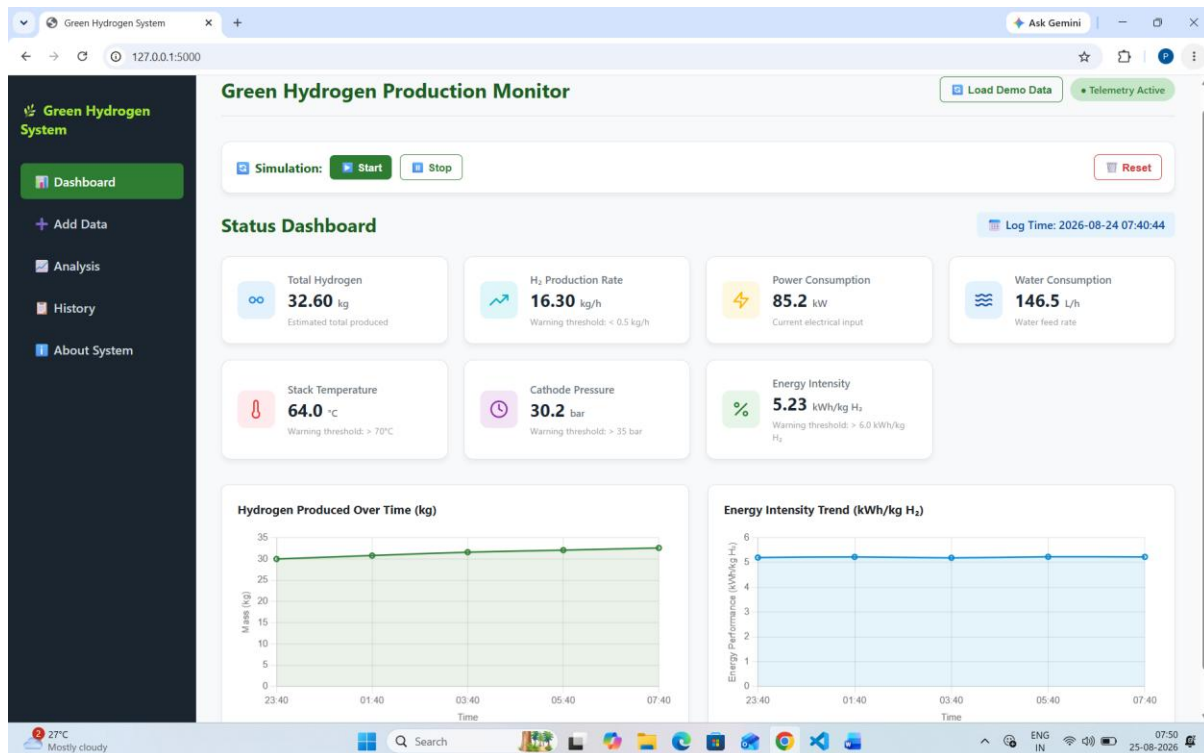


Fig 1: Main dashboard

1. Simulation & Telemetry Controls (Top)

- **Interactive Simulation:** Allows you to Start, Stop, or Reset simulated production runs.
- **Data Integration:** Features a button to Load Demo Data and an status indicator showing that live telemetry is active.

2. Live Status Metrics (Middle Cards)

- **Production Outputs:**
 - **Total Hydrogen:** The cumulative mass of hydrogen produced (currently 32.60 kg).
 - **H₂ Production Rate:** The current speed of generation (currently 16.30 kg/h, with a safety warning threshold for rates < 0.5 kg/h).
- **Resource Consumption:**

- **Power Consumption:** The electrical input power driving the reaction (currently 85.2 kW).
- **Water Consumption:** The rate of water feed required (currently 146.5 L/h).
- **System Health & Safety:**
 - **Stack Temperature:** Operating temperature of the cells (currently 64.0 °C, with a warning trigger above 70 °C).
 - **Cathode Pressure:** Pressure inside the production chamber (currently 30.2 bar, with a warning trigger above 35 bar).
- **Efficiency (Energy Intensity):**
 - Displays the energy consumed per unit of hydrogen produced (currently 5.23 kWh/kg H₂). Lower is more efficient, with a warning threshold at > 6.0 kWh/kg H₂.

3. Trend Charts (Bottom)

- **Hydrogen Produced Over Time:** A line graph displaying how the total hydrogen mass has accumulated over the last several hours.
- **Energy Intensity Trend:** Tracks the stability of the system's efficiency rate over time.

4. Navigation Menu (Left Sidebar)

- Allows switching between this **Dashboard**, an **Add Data** page (for uploading new telemetry), an **Analysis** tool, a **History** log viewer, and an **About System** information page.

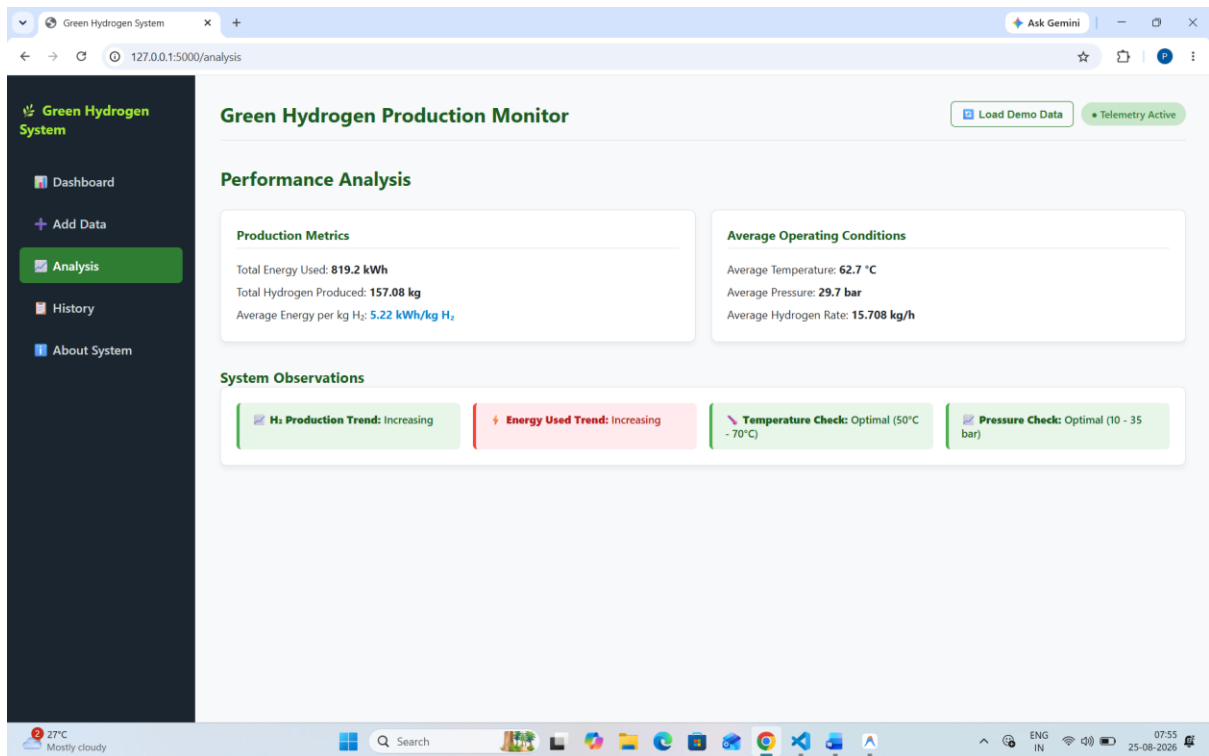


Fig 2: Analysis dashboard

This is the **Performance Analysis** page of the application. It takes the raw, real-time data from the dashboard and summarizes it into key performance indicators (KPIs) and automated safety checks.

Here is what is on the screen and what it is used for:

What it displays:

1. Production Metrics:

- Compiles cumulative statistics (Total Energy Used: 819.2 kWh, Total Hydrogen Produced: 157.08 kg) and calculates the system's average efficiency (5.22 kWh/kg H₂).

2. Average Operating Conditions:

- Calculates the mean values for temperature (62.7 °C), pressure (29.7 bar), and production rate (15.708 kg/h) over the entire logging period.

3. System Observations:

- Automated, color-coded health alerts showing whether the production and energy trends are rising/falling, and whether temperature and pressure ranges are **Optimal** or require attention.

What is the use of this page?

- **Evaluating System Efficiency:** By tracking **Average Energy per kg H₂**, operators can measure the economic viability of the system (how much electricity is needed to make a kilogram of hydrogen).
- **Preventative Safety Monitoring:** Instead of watching a fluctuating live feed, this page aggregates the average temperature and pressure. The **Temperature & Pressure Checks** instantly alert operators if the system is drifting outside of safe/optimal operational limits, preventing equipment wear or dangerous failures.
- **Trend Assessment:** It identifies whether production or energy usage is ramping up, helping managers plan electricity costs and predict future hydrogen output.
- **Simplified Decision Making:** The color-coded tags (green for optimal/stable, red for rising consumption) allow an operator to check the system's health at a single glance without having to analyze complex spreadsheets of raw data.